Contenido

- Introducción
- ¿Por qué usar PDO?
- Conexión a MySQL con PDO
- Operaciones CRUD
- CRUD con consultas parametrizadas
- Gestión de errores y buenas prácticas
- Funciones útiles en PDO
- **Buenas prácticas**
  - Prepared statements
  - Gestión de excepciones y errores
  - Seguridad en credenciales
  - Cierre de conexión
  - Modos de fetch

Acceso a bases de datos en PHP 8.3 con PDO

La manera más moderna, potente y segura de conectarse a bases de datos en **PHP 8.3** es usando la extensión **PDO** ("PHP Data Objects"). Esta interfaz permite trabajar con diversas bases (MySQL, MariaDB, SQLite, PostgreSQL...), utiliza **parámetros y prepared statements** y facilita migrar proyectos a otros motores sin cambiar el grueso del código.

¿Por qué usar PDO?

- Compatible con muchos servidores de bases de datos, no solo MySQL.
- Permite **prepared statements** para evitar inyecciones SQL.
- Gestión moderna de errores con excepciones (`try...catch`).
- API orientada a objetos, más limpia y flexible.
- Recomendado en todas las versiones modernas de PHP (MySQLi es específico de MySQL y no tan portable).

Nota: Las extensiones `mysql_*` están totalmente obsoletas y **NUNCA** se deben usar en proyectos nuevos.

Conexión a MySQL con PDO (PHP 8.3)

El siguiente ejemplo muestra la forma **más óptima, limpia y segura** para conectarse a una base de datos MySQL desde PHP usando PDO:

```php
<?php
$host = 'localhost';
$db   = 'nombre_base';
$user = 'usuario';
$pass = 'contraseña';
$charset = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";

try {
    $pdo = new PDO($dsn, $user, $pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]);
    echo "¡Conexión exitosa!";
} catch (PDOException $e) {
    echo 'Error de conexión: ' . $e->getMessage();
}
?>
```

- Siempre usa **try/catch** para manejar los posibles errores.
- El DSN se puede adaptar para otros motores solo cambiando el inicio ("mysql:...").

Operaciones CRUD con PDO

1. Crear una tabla

```php
<?php
$sql = "CREATE TABLE IF NOT EXISTS usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB";
$pdo->exec($sql);
?>
```

2. Insertar registros

```php
<?php
$stmt = $pdo->prepare("INSERT INTO usuarios (nombre, email) VALUES (?, ?)");
$stmt->execute(['Ana', 'ana@ejemplo.com']);
echo "Registro insertado con ID: " . $pdo->lastInsertId();
?>
```

3. Consultar datos (SELECT)

```php
<?php
$stmt = $pdo->query("SELECT id, nombre, email FROM usuarios");
while ($fila = $stmt->fetch()) {
    echo "{$fila['id']} - {$fila['nombre']} ({$fila['email']})<br>";
}
?>
```

4. Actualizar datos

```php
<?php
$stmt = $pdo->prepare("UPDATE usuarios SET nombre = ? WHERE id = ?");
$stmt->execute(['Ana Actualizada', 1]);
echo "Filas modificadas: " . $stmt->rowCount();
?>
```

5. Eliminar registros

```php
<?php
$stmt = $pdo->prepare("DELETE FROM usuarios WHERE id = ?");
$stmt->execute([1]);
echo "Filas eliminadas: " . $stmt->rowCount();
?>
```

CRUD con consultas parametrizadas (PDO)

Insertar registro

```php
<?php
$nombre = "Ana";
$email = "ana@ejemplo.com";
$salario = 1300.50;

$stmt = $pdo->prepare("INSERT INTO usuarios (nombre, email, salario) VALUES (?, ?, ?)");
$stmt->execute([$nombre, $email, $salario]);
echo "Registro insertado con ID: " . $pdo->lastInsertId();
?>
```

Protege contra inyecciones SQL y permite insertar valores dinámicos, usando placeholders ? en la sentencia.

Consultar registros

```php
<?php
$salarioMin = 1200;
$stmt = $pdo->prepare("SELECT id, nombre, email, salario FROM usuarios WHERE salario > ?");
$stmt->execute([$salarioMin]);
while ($fila = $stmt->fetch()) {
    echo "{$fila['nombre']} ({$fila['email']}) - Salario: {$fila['salario']}";
}
?>
```

Consulta segura filtrando por salario, el parámetro `$salarioMin` lo puede poner el usuario.

Actualizar usuario (múltiples campos)

```php
<?php
$id = 5;
$nuevoNombre = "Juan Actualizado";
$nuevoEmail = "juan.nuevo@example.com";
$nuevoSalario = 2200.50;

// Consulta previa (opcional)
$stmt = $pdo->prepare("SELECT * FROM usuarios WHERE id = ?");
$stmt->execute([$id]);
$usuario = $stmt->fetch();

if ($usuario) {
    $stmt = $pdo->prepare("
        UPDATE usuarios SET
            nombre = :nombre,
            email = :email,
            salario = :salario
        WHERE id = :id
    ");
    $stmt->execute([
        ':nombre' => $nuevoNombre,
        ':email'  => $nuevoEmail,
        ':salario'=> $nuevoSalario,
        ':id'     => $id
    ]);
    if ($stmt->rowCount() > 0) {
        echo "Actualización realizada correctamente.";
    } else {
        echo "No se realizaron cambios.";
    }
} else {
    echo "El usuario no existe.";
}
?>
```

Actualiza múltiples campos por el `id` de usuario, usando parámetros para todos.

Eliminar registro

```php
<?php
$id = 3;
$stmt = $pdo->prepare("DELETE FROM usuarios WHERE id = ?");
$stmt->execute([$id]);
echo "Filas borradas: " . $stmt->rowCount();
?>
```

Borra el usuario que coincida con el `id` indicado. Siempre seguro con placeholders.

Gestión de errores y recomendaciones

Consejos:

- Prepara siempre las consultas usando **prepared statements** (`$pdo->prepare`).
- No muestres los errores exactos en producción, solo los generales.
- No guardes contraseñas en texto plano. Usa `password_hash` y `password_verify` para almacenarlas y verificarlas.
- Cierra la conexión (destruye el objeto) cuando termines: `$pdo = null;`.

Funciones útiles en PDO y consultas

`rowCount()`
Devuelve el número de filas afectadas por la última consulta **INSERT**, **UPDATE** o **DELETE** realizada con ese statement.
Muy útil para saber si una actualización o borrado funcionó.

```php
<?php
$stmt = $pdo->prepare("DELETE FROM usuarios WHERE id = ?");
$stmt->execute([$id]);
if ($stmt->rowCount() > 0) {
    echo "Usuario eliminado.";
} else {
    echo "No existía ningún usuario con ese id.";
}
?>
```

`lastInsertId()`
Después de un **INSERT** en una tabla con clave primaria AUTO_INCREMENT, devuelve el id asignado al nuevo registro.

```php
<?php
$stmt = $pdo->prepare("INSERT INTO usuarios (nombre, email, salario) VALUES (?, ?, ?)");
$stmt->execute([$nombre, $email, $salario]);
echo "ID nuevo usuario: " . $pdo->lastInsertId();
?>
```

`fetch()`, `fetchAll()`
Permiten recuperar los resultados de un **SELECT**. `fetch()` obtiene solo una fila, `fetchAll()` recupera todas en un array.

```php
<?php
$stmt = $pdo->prepare("SELECT * FROM usuarios");
$stmt->execute();
$fila = $stmt->fetch();        // Una sola fila
$todos = $stmt->fetchAll();    // Todas las filas (array)
?>
```

`errorInfo()`
Devuelve información sobre el último error ocurrido con la consulta o conexión.
Indicado para depuración avanzada.

```php
<?php
if (!$stmt->execute([$id])) {
    print_r($stmt->errorInfo());
}
?>
```

Más funciones útiles: Revisa la documentación oficial de PDO para ver todos los métodos disponibles, ejemplos y buenas prácticas.

Buenas Prácticas al trabajar con PHP y MySQL (PDO)

1. Uso correcto de prepared statements

1. Siempre usa prepared statements en tus consultas SQL:

- Evita la concatenación de variables directamente en las sentencias.
- Ejemplo seguro:

```php
<?php
// Inseguro: $sql = "SELECT * FROM usuarios WHERE email = '$email'";
$sql = "SELECT * FROM usuarios WHERE email = ?";
$stmt = $pdo->prepare($sql);
$stmt->execute([$email]);
?>
```

2. Manejo de excepciones y errores

2. Maneja siempre los errores con `try/catch`. Operaciones que pueden generar excepciones y tipos a gestionar:

- **Operaciones que pueden lanzar excepciones (tipo `PDOException`):**
  - Creación de la conexión `new PDO()` (por error de credenciales, DSN incorrecto, base de datos no existente...)
  - Ejecutar `prepare()`, `execute()`, `query()` si el SQL está mal escrito, tabla o campo no existe, violación de restricciones (clave única, etc.).
  - Consultar datos si hay problemas de comunicación con la base de datos.
  - Comandos dentro de una transacción (`beginTransaction`, `commit`, `rollBack`).
- **Tipos de excepciones a gestionar:**
  - **PDOException**: Exclusivo de PDO, siempre que haya un error en operaciones de base de datos.
  - **Exception**: Puede gestionar cualquier excepción genérica si agrupas varios tipos.
- **Ejemplo práctico:**

```php
<?php
try {
    $pdo = new PDO($dsn, $user, $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $stmt = $pdo->prepare("INSERT INTO usuarios (nombre, email) VALUES (?, ?)");
    $stmt->execute(['Juan', 'juan@example.com']); // Aquí podría saltar excepción por email duplicado, etc.

} catch (PDOException $e) {
    // Errores de base de datos
    echo "Error de base de datos: " . $e->getMessage();
} catch (Exception $e) {
    // Otros errores genéricos
    echo "Error general: " . $e->getMessage();
}
?>
```

Consejo: En desarrollo, muestra el mensaje completo para depurar. En producción, muestra solo un texto genérico y registra el error en un log seguro.

Ejemplos de operaciones que generan excepciones en PDO

- **1. Error de conexión (credenciales incorrectas, base no existe):**

```php
<?php
try {
    $pdo = new PDO("mysql:host=localhost;dbname=noexiste", "wronguser", "wrongpass");
} catch (PDOException $e) {
    echo "Error de conexión: " . $e->getMessage();
}
?>
```

Mensaje típico: *"SQLSTATE[HY000] [1049] Base de datos desconocida 'noexiste'"*

- **2. SQL mal formado (error de sintaxis):**

```php
<?php
try {
    $stmt = $pdo->query("SELEC nombre FROM usuarios"); // error 'SELECT'
} catch (PDOException $e) {
    echo "Error en consulta SQL: " . $e->getMessage();
}
?>
```

Mensaje típico: *"SQLSTATE[42000]: Error de sintaxis cerca de 'SELEC nombre...'"*

- **3. Intentar insertar un registro que viola una restricción única (duplicado):**

```php
<?php
try {
    $stmt = $pdo->prepare("INSERT INTO usuarios (email) VALUES (?)");
    $stmt->execute(['existing@email.com']); // email ya existe, UNIQUE KEY
} catch (PDOException $e) {
    echo "Error de restricción: " . $e->getMessage();
}
?>
```

Mensaje típico: *"SQLSTATE[23000]: Integrity constraint violation: 1062 Duplicate entry... for key 'email'"*

- **4. Acceder a una tabla o campo que no existen:**

```php
<?php
try {
    $stmt = $pdo->query("SELECT campoqueNoExiste FROM usuarios");
} catch (PDOException $e) {
    echo "Error: " . $e->getMessage();
}
?>
```

Mensaje típico: *"SQLSTATE[42S22]: Column not found: 1054 Unknown column..."*

- **5. Fallos durante transacciones (commit/rollback):**

```php
<?php
try {
    $pdo->beginTransaction();
    $pdo->exec("UPDATE usuarios SET email = 'fail@email.com' WHERE id = 1");
    // Aquí puede ocurrir cualquier error de datos, etc.
    $pdo->commit();
} catch (Exception $e) {
    $pdo->rollBack();
    echo "Error durante la transacción: " . $e->getMessage();
}
?>
```

Recuerda: Para que PDO lance excepciones, siempre configura `PDO::ATTR_ERRMODE` a `PDO::ERRMODE_EXCEPTION`.
En producción, nunca muestres el mensaje completo al usuario final. Registra los errores en un log para su análisis.

3. Seguridad en el manejo de credenciales

3. Nunca incluyas credenciales reales en tu código fuente.

- Guarda contraseñas y datos sensibles en ficheros fuera del directorio público (`config.php` o variables de entorno).
- Mantén tus archivos `config.php` fuera del alcance de los navegadores y **no los subas a repositorios públicos**.

4. Cierre de la conexión a base de datos

4. Cierre de la conexión a base de datos con PDO

Cuando ya no necesitas realizar más consultas, puedes liberar la conexión simplemente asignando `$pdo = null;`. Aunque PHP destruye el objeto automáticamente al terminar el script, hacerlo explícitamente es buena práctica en scripts largos o con muchas conexiones.

```php
<?php
// Crear conexión PDO al principio
$pdo = new PDO($dsn, $usuario, $contraseña);

// ... realizar consultas ...

// Cerrar la conexión explícitamente
$pdo = null;
?>
```

5. Modos de obtención de resultados en PDO (`fetch modes`)

5. Modos de obtención de resultados en PDO (`fetch modes`)

Cuando consultamos datos con PDO, **por defecto** el modo es `PDO::FETCH_BOTH` (puedes acceder por índice y por nombre de campo). Sin embargo, en buenas prácticas usamos otros modos según la necesidad:

`PDO::FETCH_ASSOC` (recomendado): Devuelve cada fila como un **array asociativo** usando los nombres de campo.

```php
<?php
$stmt = $pdo->query("SELECT * FROM usuarios");
while($fila = $stmt->fetch(PDO::FETCH_ASSOC)) {
    echo $fila['nombre'];  // Acceso por nombre de columna
}
?>
```

`PDO::FETCH_OBJ`: Devuelve la fila como un **objeto anónimo**, accediendo con ->.

```php
<?php
$stmt = $pdo->query("SELECT * FROM usuarios");
while($obj = $stmt->fetch(PDO::FETCH_OBJ)) {
    echo $obj->nombre;
}
?>
```